

TRD — Arquitetura Técnica

Gestão de Materiais Corporativo

Gerado em 26/09/2026

Fonte: TRD.md

1. Decisão técnica

O sistema será um monólito modular com frontend desacoplado:

```
Next.js → experiência web
Django + DRF → API e regras de negócio
PostgreSQL → persistência transacional
Redis + Celery → jobs e tarefas assíncronas
FastAPI → serviço de IA futuro
```

O monólito modular reduz complexidade inicial e mantém fronteiras claras para extrair serviços quando houver necessidade real.

2. Repositório

```
frontend/      Next.js, React, TypeScript
backend/       Django, DRF, apps de domínio
ai-service/    FastAPI futuro
infra/         deploy e serviços externos
docs/          documentação
.github/       CI
```

3. Backend

Stack

- Python 3.11+;
- Django 5.2+;
- Django REST Framework;
- SimpleJWT;
- PostgreSQL;
- Celery;
- Redis;
- Pytest e pytest-django.

Apps de domínio

```
apps/accounts
apps/organizations
apps/materials
apps/inventory
apps/requests
apps/purchasing      # Sprint 06
apps/deliveries      # Sprint 07
apps/assets          # Sprint 07
apps/auditing        # Sprint 08
apps/reports         # Sprint 09
```

Camadas

```
api_views.py      HTTP, autenticação e serialização de entrada
serializers.py    contratos e validações de transporte
services.py       casos de uso e transações
models.py         persistência e invariantes locais
selectors.py      consultas de leitura complexas
permissions.py    autorização e escopo
```

Regras de negócio não devem ser implementadas somente nas views.

4. Frontend

Stack

- Next.js App Router;
- React;
- TypeScript;
- Tailwind CSS;
- Vitest e Testing Library;
- Playwright para fluxos críticos.

Princípios

- Server Components por padrão;
- Client Components somente quando houver interação;
- API Django como fonte de verdade;
- Nunca confiar em permissões escondendo botão;
- Estados de loading, erro, vazio e sucesso explícitos;
- Tipos compartilhados ou contratos gerados quando a API crescer.

5. Banco de dados

Entidades já implementadas

```
organizations
areas
users
roles
user_roles
material_categories
materials
warehouses
stock_balances
stock_movements
stock_reservations
material_requests
request_items
request_approvals
request_status_history
```

Entidades futuras

```
branches
departments
suppliers
purchase_requests
quotations
quotation_items
purchase_orders
purchase_order_items
receipts
invoices
assets
custodies
custody_terms
maintenance_orders
inventories
```

```
inventory_items
audit_logs
attachments
notifications
```

Invariantes

- SKU é único dentro da organização;
- Código de categoria é único dentro da organização;
- Código de almoxarifado é único dentro da organização;
- Saldo físico e reservado não podem ser negativos;
- Quantidade de movimento é positiva;
- Chave de idempotência é única;
- Solicitação aprovada não pode ser aprovada novamente;
- Relações históricas usam PROTECT quando necessário.

6. Contratos de API

Resposta de sucesso de recurso:

```
{
  "data": {},
  "meta": {}
}
```

Resposta de erro:

```
{
  "detail": ["Descrição do erro"]
}
```

Status esperados:

```
200 leitura/ação concluída
201 criação/movimentação criada
204 logout sem corpo
400 regra ou validação inválida
401 não autenticado
403 sem permissão
404 fora do escopo ou inexistente
409 conflito de idempotência/estado
```

Endpoints atuais:

```
POST /api/auth/token/
POST /api/auth/token/refresh/
POST /api/auth/logout/
GET /api/auth/me/

POST /api/inventory/receive/
POST /api/inventory/issue/
POST /api/inventory/reserve/
POST /api/inventory/release/
POST /api/inventory/transfer/

GET /api/requests/
POST /api/requests/
GET /api/requests/{id}/
POST /api/requests/{id}/submit/
POST /api/requests/{id}/approve/
POST /api/requests/{id}/reject/
```

7. Segurança

- Senha com hash do Django;
- JWT com refresh blacklist;
- HTTPS em produção;
- CORS explícito;
- Segredos somente em ambiente;
- Validação de organização em toda mutação;
- Permissões no backend;
- Uploads validados por tipo e tamanho;
- Logs sem senhas ou tokens;
- MFA para administradores como etapa futura;
- Rate limiting de login como etapa de hardening;
- Auditoria de ações críticas;
- Princípio do menor privilégio.

8. Concorrência e integridade

Operações de estoque devem usar:

```
transaction.atomic()  
select_for_update()
```

O saldo não deve ser alterado por endpoint genérico. Toda alteração deve passar por um caso de uso de estoque.

Operações com efeitos externos devem ser idempotentes e, quando houver integração, usar padrão outbox ou chave de processamento.

9. Jobs assíncronos

Celery será utilizado para:

- Notificações;
- Importações;
- Relatórios pesados;
- Alertas de estoque mínimo;
- OCR;
- Chamadas ao serviço de IA;
- Rotinas de reconciliação.

Jobs devem ser:

- Idempotentes;
- Observáveis;

- Reexecutáveis;
- Com timeout;
- Com número máximo de tentativas;
- Sem duplicar efeitos transacionais.

10. Deploy

```
GitHub → CI
Vercel → frontend
Railway/Render/Fly.io → backend e worker
Neon/Supabase → PostgreSQL
Upstash → Redis
S3/R2/Supabase Storage → arquivos
```

Ambientes:

```
dev
homologação
produção
```

Cada ambiente terá banco, segredos e arquivos isolados.

11. Observabilidade

Futuro obrigatório:

- Health check;
- Logs estruturados;
- Correlation ID;
- Métricas de latência;
- Erros por endpoint;
- Falhas de jobs;
- Auditoria de movimentações;
- Alertas de indisponibilidade;
- Monitoramento de banco e fila.

12. Testes

Backend

- Testes de modelo;
- Testes de serviço;
- Testes de API;
- Testes de autorização;
- Testes de concorrência para estoque;
- Testes de idempotência;
- Testes de migrations.

Frontend

- Testes de componentes;
- Testes de estados de erro;
- Testes de formulários;
- Testes E2E de login, solicitação e aprovação.

Gate mínimo:

```
cd backend
.venv/Scripts/python.exe -m pytest -q
.venv/Scripts/python.exe manage.py check

cd ../frontend
npm run lint
npm run build
```